School of Electronics and Computer Science
University of Southampton

The Perceptron

1 Teaching a Perceptron Logical AND

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Step 1: data and plotting helpers. Copy this code. You will reuse `plot_boundary` throughout the lab: it shades the regions that any classifier assigns to each class.

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
t = np.array([0, 0, 0, 1])                          # AND targets

def plot_boundary(predict, X, t, title=""):
    xx, yy = np.meshgrid(np.linspace(X[:, 0].min() - 0.5, X[:, 0].max() + 0.5, 300),
                         np.linspace(X[:, 1].min() - 0.5, X[:, 1].max() + 0.5, 300))
    zz = predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    plt.contourf(xx, yy, zz, levels=[-0.5, 0.5, 1.5], colors=["#ffcccc", "#cce0ff"])
    plt.scatter(X[t == 0, 0], X[t == 0, 1], c="red", marker="o", label="Class 0")
    plt.scatter(X[t == 1, 0], X[t == 1, 1], c="blue", marker="x", s=80, label="Class 1")
    plt.xlabel("$x_1$"); plt.ylabel("$x_2$"); plt.title(title); plt.legend(); plt.show()
```

Step 2: complete the perceptron. Fill in the three `TODO` lines using the recap box above.

```
def add_bias(X):
    return np.hstack([np.ones((len(X), 1)), X])       # adds the x0 = 1 column

def perceptron_fit(X, t, eta=1.0, w_init=None, max_epochs=100):
    Xb = add_bias(X)
    w = np.zeros(Xb.shape[1]) if w_init is None else np.array(w_init, dtype=float)
    mistakes_per_epoch = []
    for epoch in range(max_epochs):
        mistakes = 0
        for x, target in zip(Xb, t):
            y = ...                                    # TODO: 1 if w @ x > 0, else 0
            if y != target:
                w = ...                                # TODO: the update rule
                mistakes += 1
        mistakes_per_epoch.append(mistakes)
        if ...:                                        # TODO: stop when converged
            break
    return w, mistakes_per_epoch

def perceptron_predict(w, X):
    return (add_bias(X) @ w > 0).astype(int)
```

Step 3: train and inspect.

1. Train with `w, history = perceptron_fit(X, t)`. Print the final weights and the number of epochs.
2. Plot the boundary: `plot_boundary(lambda P: perceptron_predict(w, P), X, t)`.
3. Plot `history` (mistakes per epoch).
4. **Learning rate.** Fill in this table with the number of epochs needed:

Starting weights	$\eta = 0.01$	$\eta = 0.1$	$\eta = 1$
all zeros (`w_init=None`)			
$(1, 0, 0)$ (`w_init=[1, 0, 0]`)			

Questions to think about

What weights did the perceptron learn, and what does the boundary look like? Does the learning rate change the number of epochs when the weights start at zero? What about when they start at $(1, 0, 0)$?

2 A Larger Dataset with scikit-learn

Generate two classes of 100 points each from 2D Gaussian distributions, and keep 30% of the data aside for validation:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron

rng = np.random.default_rng(0)                        # use this seed
C = np.array([[2, 1], [1, 2]])                        # shared covariance
data0 = rng.multivariate_normal(mean=[0, 3], cov=C, size=100)   # class 0
data1 = rng.multivariate_normal(mean=[3, 0], cov=C, size=100)   # class 1
X2 = np.vstack([data0, data1])
t2 = np.r_[np.zeros(100, dtype=int), np.ones(100, dtype=int)]

X_tr, X_va, t_tr, t_va = train_test_split(X2, t2, test_size=0.3,
                                          stratify=t2, random_state=0)
```

1. Train `clf = Perceptron(random_state=0)` with `clf.fit(X_tr, t_tr)`.
2. Report the accuracy on the training and validation sets with `clf.score(...)`.
3. Plot the boundary with `plot_boundary(clf.predict, X_tr, t_tr)`.
4. Train *your own* `perceptron_fit` on `X_tr, t_tr` with `max_epochs=1000`. Plot its mistakes per epoch. Does it reach zero mistakes?

Questions to think about

How accurate is the perceptron on the training and validation data? Look at your plot: can a straight line separate the two classes perfectly? Use this to explain the behaviour of your own perceptron in step 4.

3 Trying Different Activation Functions

Instead of the step function, a neuron can use a smooth activation and learn by **gradient descent**. The function below does this for you. Your job is to supply the activations.

```
def sigmoid(z): return ...      # TODO
def tanh(z):    return ...      # TODO
def relu(z):    return ...      # TODO

ACTIVATIONS = {  # name: (f, f'(a, z), targets for class 0/1, threshold)
    "sigmoid": (sigmoid, lambda a, z: a * (1 - a),           (0, 1),  0.5),
    "tanh":    (tanh,    lambda a, z: 1 - a**2,              (-1, 1), 0.0),
    "relu":    (relu,    lambda a, z: (z > 0).astype(float), (0, 1),  0.5),
}

def neuron_fit(X, t, act, eta=0.1, epochs=3000):
    f, df, (lo, hi), _ = ACTIVATIONS[act]
    target = np.where(t == 1, hi, lo)               # targets in this activation's range
    Xb = add_bias(X)
    w = np.random.default_rng(0).normal(0, 0.1, Xb.shape[1])   # small random start
    for epoch in range(epochs):
        z = Xb @ w
        a = f(z)
        w = w - eta * Xb.T @ ((a - target) * df(a, z)) / len(t)
    return w

def neuron_predict(w, X, act):
    f, _, _, threshold = ACTIVATIONS[act]
    return (f(add_bias(X) @ w) > threshold).astype(int)
```

1. Fill in the three activation functions: sigmoid $1/(1+e^{-z})$, tanh (use `np.tanh`) and ReLU $\max(0, z)$ (use `np.maximum`).
2. Train each activation on the Gaussian training set from Part 2, and record the time taken and the training and validation accuracy. (Tip: `time.perf_counter()` gives the current time in seconds.)

Activation	Time (ms)	Train acc.	Validation acc.
step (Part 2)			
sigmoid			
tanh			
ReLU			

3. Plot the decision boundary for each activation.

Questions to think about

Which activation gave the best validation accuracy? Is the decision boundary still a straight line? Why do you think that is?

Optional: Real Data

Try the perceptron on a real medical dataset: classifying breast tumours as malignant or benign from 30 measurements.

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X3, t3 = load_breast_cancer(return_X_y=True)
```

```
X_tr3, X_te3, t_tr3, t_te3 = train_test_split(X3, t3, test_size=0.3,
                                              stratify=t3, random_state=0)
raw    = Perceptron(random_state=0).fit(X_tr3, t_tr3)
scaled = make_pipeline(StandardScaler(), Perceptron(random_state=0)).fit(X_tr3, t_tr3)
```

Compare the test accuracy of `raw` and `scaled`. Why does scaling the features help? What accuracy would you get by always predicting the most common class?